

DESCRIEREA SOLUȚIILOR, OLIMPIADA NAȚIONALĂ DE INFORMATICĂ CLASA A VIII-A

COMISIA ȘTIINȚIFICĂ

PROBLEMA 1: PROEMINENȚA

Propusă de: Lector dr. Paul Diac, Facultatea de Informatică, Universitatea „Alexandru Ioan Cuza” Iași

Pentru ambele cerințe trebuie eliminate altitudinile consecutive egale. Acest lucru se poate face prin citirea altitudinilor într-o variabilă temporară x și adăugarea în vector doar a valorilor diferite de ultima adăugată:

```
if (x != a[n-1]) { a[n++] = x; }
```

Pentru cerința $C = 1$:

Se parcurg altitudinile $0 < i < n - 1$ și se numără vârfurile (pe poziția i este un vârf dacă $a[i - 1] < a[i] > a[i + 1]$).

Pentru cerința $C = 2$:

Soluție $O(N^3)$. O primă soluție pentru a determina proeminența unui vârf de pe poziția i este să aplicăm definiția în mod direct. Parcurgem orice alt vârf j și le procesăm pe cele cu $a[j] > a[i]$. Pentru a determina cea mai adâncă vale dintre i și j , se parcurg pozițiile k dintre $\min(i, j)$ și $\max(i, j)$ și se reține cea mică valoare $a[k]$. Proeminența vârfului i va fi diferența minimă dintre $a[i]$ și valorile $a[k]$ obținute astfel. Soluția se încadrează în timp pe testele cu $N \leq 1000$.

Soluție $O(N^2)$. După ce fixăm un vârf i pentru care calculăm proeminența, putem parcurge vârfurile j începând de la poziția i prima dată în stânga, apoi în dreapta. Prin aceste parcurgeri putem determina vârfuri $a[j] > a[i]$ concomitent cu a determina cea mai adâncă vale dintre ele. Reținem această altitudine minimă într-o variabilă pe care o folosim și pentru următoarele poziții j .

Optimizare: Analizând mai atent definiția proeminenței, deducem că pe un profil proeminența unui vârf i va fi dată de **primul** vârf mai mare decât $a[i]$ situat în stânga sa $a[s] > a[i]$ ($s < i$) sau de **primul** vârf mai mare decât $a[i]$ situat în dreapta sa $a[d] > a[i]$ ($d > i$). Justificarea constă în faptul că, dacă ar exista alte vârfuri $a[k] > a[i]$ cu $k < s$ sau $k > d$ până la care văile ar fi și mai adânci, ele nu vor fi relevante, deoarece atunci când se alege minimul dintre diferența $a[i]$ și cea mai adâncă vale, ele dau o diferență mai mare, care nu modifică minimul. Soluțiile de complexitate $O(N^2)$ se încadrează în timp pe un număr semnificativ mai mare de teste, în funcție de optimizările efectuate.

Soluție $O(N)$. Pentru a proiecta o soluție de complexitate liniară determinăm mai întâi „proeminența la stânga”, parcurgând vârfurile de la stânga către dreapta. Procedăm apoi similar pentru a determina „proeminența la dreapta”, parcurgând vârfurile de la dreapta către stânga și la final afișăm minimul dintre cele două valori obținute pentru proeminență.

Pentru a calcula *proeminența la stânga*, parcurgem vârfurile în ordinea crescătoare a indicilor i . Pentru orice vârf i doar cel mai apropiat vârf j din stânga lui i ($j < i$), strict mai înalt decât vârful i este relevant, prin urmare toate vârfurile dintre acestea $j < k < i$ care au altitudini mai mici $a[k] < a[i]$, pot fi ignorate mai departe. Este deci suficient să reținem o **stivă** cu vârfuri montane în ordinea strict descrescătoare a înălțimilor. Pentru fiecare dintre aceste vârfuri, vom reține și cea mai adâncă vale situată în dreapta vârfului, până la următorul vârf adăugat în stivă, vale pe care o determinăm „din mers”.

Rezumând, parcurgem profilul de la stânga la dreapta și identificăm vârfurile montane (fie $a[i]$ altitudinea vârfului montan curent).

Cât timp în stivă există elemente și $a[i] \geq$ altitudinea vârfului montan plasat la vârful stivei, extragem din stivă un element (evident, doar elementul situat la vârful stivei poate fi extras). Atunci când extragem un element din stivă, actualizăm, dacă este cazul, și înălțimea celei mai adânci văi până la vârful montan curent.

Dacă stiva a devenit vidă, inserăm vârful montan curent în stivă și inițializăm înălțimea celei mai adânci văi până la momentul curent cu $a[i]$.

Dacă stiva este nevidă, atunci $a[i] <$ altitudinea vârfului plasat la vârful stivei. În acest caz:

- (1) pentru vârful montan situat în vârful stivei memorăm cea mai mică altitudine a unei văi identificate până la vârful montan curent;
- (2) determinăm proeminența la stânga a vârfului montan curent ca fiind diferența dintre $a[i]$ și altitudinea minimă a unei văi până la vârful montan curent;
- (3) reinițializăm înălțimea minimă a văii curente cu $a[i]$;
- (4) inserăm vârful montan curent în stivă.

Pentru a evita duplicarea codului pentru „*proeminența la dreapta*”, putem oglindi vectorul și repeta aceeași procedură.

```
#include <stdio.h>
/*
autor      Paul Diac
idee       Iteram toate inaltimele, retinem varfurile in ordine strict
           descrescatoare si cele mai adanci vai de dupa ele. Un varf nou adaugat
           elimina toate varfurile mai mici decat el, deci vom folosi o stiva
           pentru varfuri. Pentru a evita alte cazuri, la inceput
           calculam proeminenta la stanga, si apoi la dreapta dupa o oglindire.
           La final afisam minimul dintre cele doua proeminente calculate.
memorie    O(N)
timp       O(N)
scor       100
*/
#include <stdio.h>
#define NMax 1000005
long int N, C, s;
long int a[NMax];

long int min_(long int i, long int j) { if (i < j) return i; return j; }
long int peak[NMax], valley[NMax], top = -1; // peak este stiva de varfuri
                                           // valley retine cele mai adanci vai"
long int prom[2][NMax]; // proeminenta la stanga varfului de pe pozitia i este prom[0][i]
                        // si la dreapta este prom[0][n-1-i] datorita oglindirii

void popTop() { // eliminam un varf din stiva, actualizam cea mai adanca vale
               // din dreapta daca e cazul
    if (top < 0) { return; }
    if (top >= 1 && valley[top-1] > valley[top]) {
        valley[top-1] = valley[top];
    }
    top--;
}

int main() {
    freopen("proeminenta.in", "r", stdin);
    freopen("proeminenta.out", "w", stdout);

    scanf("%ld_%ld", &C, &N);
    for (int i = 0; i < N; i++) {
        scanf("%ld", &a[i]);
```

```
if (i > 0 && a[i] == a[i-1]) {
    i--; N--; // eliminam duplicatele
}
}
if (C == 1) {
    for (int i = 1; i < N-1; i++) {
        if (a[i-1] < a[i] && a[i] > a[i+1]) { s++; } // este varf
    }
    printf("%ld\n", s);
    return 0;
}

for (int d = 0; d <= 1; d++) { // d=0 va fi parcurgerea stanga-dreapta, si d=1
    // dupa oglindire, adica dreapta-stanga
    for (int i = 1; i < N-1; i++) {
        if (a[i-1] < a[i] && a[i] > a[i+1]) { // este varf
            prom[d][i] = a[i];
            while (top >= 0 && a[peak[top]] <= a[i]) { // elimim varfurile mai
                // mici din stiva pe rand
                popTop();
            }
            if (top >= 0 && a[peak[top]] > a[i]) {
                prom[d][i] = a[i] - valley[top];
                // am gasit primul varf mai mare din stanga
            }
            peak[++top] = i;
            valley[top] = a[i]; // adaugam noul varf
        }
        if (top >= 0 && valley[top] > a[i]) { valley[top] = a[i]; }
    }
    for (int i = 0; i < N / 2; i++) { // oglindim altitudinile
        long aux = a[i]; a[i] = a[N-1-i]; a[N-1-i] = aux;
    }
    top = -1;
}

for (int i = 1; i < N-1; i++) if (a[i-1] < a[i] && a[i] > a[i+1]) {
    if (prom[0][i] > prom[1][N-1-i]) { // retinem maximul in prom[0][i]
        prom[0][i] = prom[1][N-1-i];
    }
}

for (int i = 1; i < N-1; i++) {
    if (prom[0][i] > 0) { printf("%ld", prom[0][i]); } // si il afisam
}

printf("\n");
fclose(stdout);
return 0;
}
```